```
...
class VenueIn(ModelSchema):
    class Meta:
        model = Venue
        fields = ["name", "description"]

...

@router.post("/venue/", response=VenueOut, auth=api_key)
def create_venue(request, payload:VenueIn):
    venue = Venue.objects.create(**payload.dict())
    return venue
```

A new serializer specific to creating venues, which doesn't require an ID

The response from a creation action is the resulting Venue. By specifying the auth argument, you restrict access to this endpoint.

Using VenueIn as a Python type hint tells Ninja how to handle the payload body.

Venue.objects.create() expects key–value pairs for the fields in a Venue object; using the ** notation turns a dictionary into this format.

If you're concerned about how to test this, thinking you'll need to manually create an HTTP call with the appropriate header, don't worry—Ninja has your back. Similar to how Django provides the Admin, Ninja provides a tool for interacting with your API. It is called *docs*, and you invoke it by adding *docs* to the end of your API's URL. You'll recall me complaining about Ninja's lack of consistency with trailing slashes; this is one of those cases: putting a slash on the end of the URL won't work. To see the interface for your own API, visit http://localhost:8000/api/v1/docs.

Unit testing Ninja calls

Although it is great to have the Ninja docs interface to play with your API, in a real project, you're still going to want to write unit tests. In chapter 8, you saw how to use the `.client.get()` and `.client.post()` member methods of the `TestCase` class. These work equally well with Ninja. There are also methods for `.put()`, `.patch()`, and `.delete()`.

The `.post()`, `.put()`, and `.patch()` methods that are used for creating and updating content all support an optional argument: `content_type`. By setting this value to `application/json`, Django automatically converts your data argument into JSON. In your unit tests, create a Python dictionary containing the key–value pairs for your content, and then pass it into the call using the JSON content type, and Django's test framework takes care of the rest.

Another useful optional argument that is supported by all the HTTP method calls is `headers`. This value takes a dictionary with key–value pairs specifying additional headers to include in the call. This is how you test against calls guarded by key-based authentication. The following example combines both these ideas:

```
from django.conf import settings
from django.test import TestCase

class APITests(TestCase):
    def test_venue_api(self):
        headers = {
            "X-API-KEY": settings.NINJA_API_KEY,
```

A dictionary containing the API key's HTTP header